REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
UNIVERSITY OF BUEA
BUEA, SOUTH-WEST REGION
P.O BOX 63.

RÉPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
UNIVERSITÉ DE BUEA
BUEA, RÉGION DU SUD-OUEST
B.P. 63.

Group 23

FACULTY OF ENGINEERING TECHNOLOGY

Report on the

System Modelling and Design

Course Title: Internet Programming and Mobile App Development

Course code: CEF440

Course Instructor: Dr Nkemeni Valery

Date: May, 2026

Group 23 Members

NAME	MATRICULE
1. THELMA MARLEY NDAYINU	FE23A171
2. EKWEMBWE NKWELLE JUNIOR	FE23A044
3. MBUH ASSURANCE MULUH PILA	FE23A091
4. NYUH DELBERT KIMBI	FE23A129
5. YASIRI TEMWA GASTON	FE23A175

Table of Contents

1. Introduction

1.1 Background

In Cameroon, mobile network penetration has grown substantially over the past decade, with operators such as MTN Cameroon and Orange Cameroon serving millions of subscribers across urban and rural regions. Mobile data has become the primary means of internet access for the majority of Cameroonians, used daily for social media, mobile banking, e-learning, telemedicine and government service delivery.

Despite this growth, mobile subscribers in Cameroon continue to experience significant and persistent network quality challenges. Slow data speeds, dropped calls, signal instability, frequent disconnections and high latency are reported consistently across different regions and network operators. These issues are not merely inconveniences — they directly affect productivity, educational outcomes, business operations and the overall quality of life for people who depend on mobile connectivity as their primary digital infrastructure.

Mobile network operators invest considerable resources in Quality of Service (QoS) monitoring infrastructure, deploying sophisticated tools that measure network performance from the operator side. These systems monitor tower uptime, bandwidth capacity, handover success rates, and radio frequency conditions across the network. However, a critical gap exists between what these systems measure and what subscribers actually experience. QoS monitoring operates from the infrastructure perspective — it measures how the network performs technically. It does not measure how subscribers perceive and experience that performance in the context of their real-world mobile usage.

This distinction between Quality of Service and Quality of Experience is the fundamental motivation for the QoE-Monitor system. Quality of Experience (QoE) is a human-centered measure that captures the subscriber's subjective perception of service quality — how fast the internet feels, how clear a call sounds, how frustrating a buffering video is. QoE cannot be measured from a network tower. It can only be measured from the subscriber's device, in real time, during actual usage.

1.2 Problem Statement

Mobile network operators in Cameroon currently have no reliable, automated, real-time mechanism to collect Quality of Experience data directly from their subscribers. The monitoring tools they deploy measure technical infrastructure performance from the network side. They are structurally blind to what subscribers feel, perceive and experience during their daily mobile usage.

This creates a feedback gap with serious consequences. When network problems arise, operators discover them reactively, through **customer care complaints**, social media posts, or internal alarm systems triggered by infrastructure thresholds. By the time a problem is formally identified through these channels, subscribers have already experienced service degradation, potentially for hours or days. The unstructured nature of customer care complaints means operators receive limited actionable insight about where, when and why QoE is poor.

A subscriber calling to complain about slow internet provides minimal diagnostic value compared to an automated system that records the exact signal strength, network type, latency value and timestamp at the moment of degradation.

Furthermore, the Telecommunications Regulatory Authority of Cameroon (ART) faces challenges in independently verifying the QoS commitments made by operators, as all available data originates from the operators themselves. There is no independent, subscriber-side data source that can challenge operator-reported performance metrics.

Survey data collected during the requirement gathering phase of this project strongly validates the problem. Of 121 valid responses from active smartphone users across Cameroon, **89.4%** reported high daily dependence on mobile data for essential activities including Social Media at 73.8%, **Messaging at 66.4%**, and **Work and Education at 59.0%**. Despite this heavy reliance, the majority of respondents rated their overall network experience as **Poor or Very Poor**. Respondents reported that existing feedback channels, primarily operator customer care lines are slow, frustrating, and rarely lead to any noticeable improvement. A significant majority expressed willingness to install an app that monitors network quality, provided it does not drain their battery, consume excessive mobile data, or compromise their privacy.

The QoE-Monitor system is designed to directly address this problem. It is a native Android mobile application that collects both subjective user QoE feedback and objective device-level network performance metrics in real time, stores data locally using an Offline-First architecture, and synchronizes anonymized records to a cloud backend where operators can access aggregated, actionable insights. The system bridges the gap that existing QoS monitoring infrastructure leaves open, creating for the first time a subscriber-side QoE data pipeline for Cameroonian mobile network operators.

1.3 Objectives of This Report

This report presents the complete system modelling and design phase of the QoE-Monitor project, corresponding to Task 4 of the CEF440 Mobile Programming course. The objectives of this phase are to translate the requirements established in Task 3 into a formal visual and structural representation of the system using standard modelling notations.

Specifically, this report covers six diagrams that collectively define the system from multiple perspectives:

- The Context Diagram establishes the system boundary and defines all external entities and data flows at the highest level of abstraction.
- The Dataflow Diagram decomposes the system into its internal processes and shows how data moves between them and through data stores.
- The Use Case Diagram defines all system functionalities from the user perspective, showing what each actor can do with the system.
- The Sequence Diagram shows the time-ordered interaction between system components for the most critical operational scenarios.
- The Class Diagram defines the static structural architecture of the system including all classes, their attributes, methods and relationships.
- The Deployment Diagram shows the physical architecture of the system across hardware nodes and defines how those nodes communicate.

Together these diagrams provide the complete design blueprint from which the QoE-Monitor application will be built in subsequent development phases.

1.4 System Overview

QoE-Monitor is a native Android application built with Kotlin. It is targeted at the dominant Android smartphone ecosystem in Cameroon, devices running Android 8.0 (API Level 26) and above. iOS is explicitly excluded from the system scope because Apple does not expose the TelephonyManager-equivalent APIs that are required to read signal strength and network type data from the device these APIs are restricted to Apple system applications regardless of what programming language or framework is used.

The system collects two categories of data. Objective data consists of measurable network indicators automatically collected by the application in the background signal strength in decibel-milliwatts, network type such as 2G, 3G or 4G, and round-trip latency in milliseconds. Subjective data consists of user-reported experience ratings collected through event-triggered feedback prompts overall satisfaction, call quality and browsing experience rated on a five-point scale.

The system follows an Offline-First architecture. All collected data is persisted immediately to a local Room Database on the device, regardless of whether internet connectivity is available.

This is not merely a design preference but a fundamental requirement given that the system is designed to monitor network unreliability so, it cannot depend on the very resource it is measuring. When connectivity is available, anonymized records are synchronized to a Supabase cloud backend where network operators can access aggregated quality insights through read-only database views. No personally identifiable information ever leaves the device.

The monitoring architecture is intelligent and context-aware rather than continuous. True continuous background polling is not architecturally viable on modern Android devices because operating system battery optimization policies actively terminate or suspend processes that consume resources without user-visible activity. The system instead uses a hybrid event-based approach combining WorkManager for lightweight scheduled periodic checks, Broadcast Receiver for real-time network event detection, and a Foreground Service that

provides persistent background operation with a visible notification as required by Android policy. This architecture achieves the goal of continuous network awareness while remaining compatible with Android's operational constraints and user expectations around battery consumption.

2. Context Diagram

2.1 Purpose and Scope

The Context Diagram represents the highest level of system abstraction. It defines the system boundary by depicting the entire QoE-Monitor application as a single process and showing all external entities that interact with it. The diagram answers the fundamental question: who or what does the system exchange data with, and what data is exchanged? No internal logic, database structures or implementation details are shown at this level. The context diagram is concerned exclusively with the system's external interface.

For the QoE-Monitor system, the context diagram is particularly important because the system sits at the intersection of several distinct external environments: the subscriber's usage context, the Android operating system layer, the cloud backend infrastructure, and the network operator's analytics environment. Defining these boundaries clearly is essential before proceeding to more detailed modelling.

2.2 Central Process

The entire QoE-Monitor application is represented as a single central oval labeled QoE Monitoring System. This represents the complete application boundary. Everything that happens inside the application — metric collection, event detection, local storage, synchronization, dashboard rendering — is abstracted into this single process at the context level.

2.3 External Entities

Four external entities interact with the central system process. Each entity represents a distinct actor that either sends data into the system or receives data from it, or both.

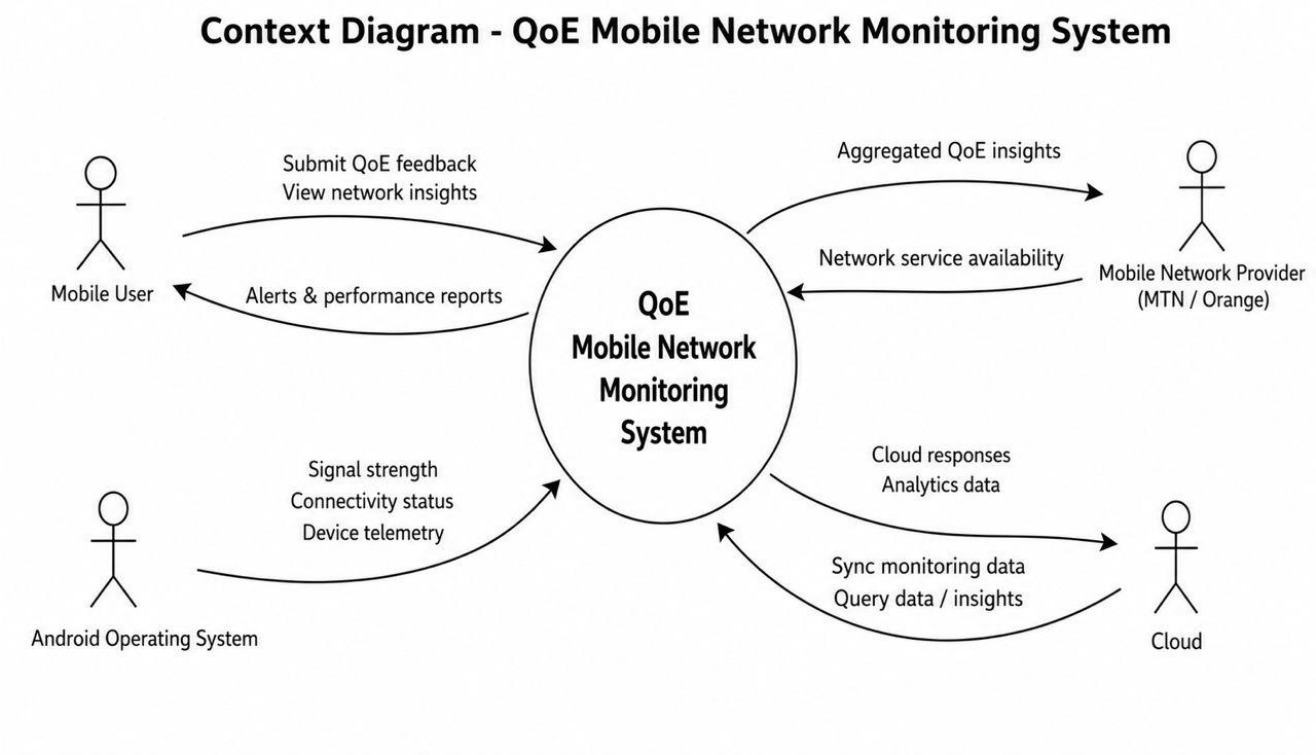
The Mobile Subscriber is the primary external entity. This represents the human user who installs and uses the QoE-Monitor application on their Android device. The subscriber sends data into the system in the form of QoE feedback ratings when responding to prompts, manual report submissions when they choose to report their experience independently, and permission approvals during the onboarding process. In return, the subscriber receives feedback prompts that notify them when the system has detected a network event and requests their rating, the personal QoE dashboard that displays their historical network experience, and general notifications about monitoring status.

Android System APIs represent the second external entity. This is not a human actor but rather the Android operating system's application programming interface layer — the set of system services through which the QoE-Monitor application accesses hardware and platform data. The Android APIs send into the system the signal strength values in dBm from TelephonyManager, network type information identifying whether the device is on 2G, 3G or 4G from TelephonyManager and ConnectivityManager, connectivity state changes and events from Connectivity Manager's Broadcast Receiver, call state transitions from PhoneStateListener, and scheduled task triggers from WorkManager. The system sends API calls back to Android APIs as requests for specific data values.

The Network Operator represents the third external entity. This entity represents MTN Cameroon, Orange Cameroon, or any other network operator who has authorized access to the QoE-Monitor backend analytics. The operator receives from the system aggregated anonymized QoE reports summarizing average satisfaction scores, network type distributions, and peak degradation time periods across all subscribers. The operator can also send report queries and analysis requests to the system through the backend interface, though these are read-only interactions with no ability to access individual subscriber records.

Supabase Backend represents the fourth external entity. This is the cloud database infrastructure that serves as the persistent storage destination for synchronized data. The system sends anonymized records to Supabase via HTTPS REST API calls whenever WiFi connectivity is available. Supabase returns upload confirmation responses that the system uses to update the synchronization status of local records.

2.3.1 Context Diagram



2.4 Data Flows Summary

Data Flow	Direct	Between	Data Exchanged
Subscriber Input	Inbound	Mobile Subscriber → System	QoE ratings, feedback responses, permission approvals
Subscriber Output	Outbound	System → Mobile Subscriber	Feedback prompts, QoE dashboard, notifications
API Input	Inbound	Android APIs → System	Signal strength, network type, call state, connectivity events
API Requests	Outbound	System → Android APIs	API calls requesting telephony and connectivity data
Operator Output	Outbound	System → Network Operator	Aggregated QoE reports and network trend analytics

Data Flow	Direct	Between	Data Exchanged
Operator Input	Inbound	Network Operator → System	Report queries and analysis requests
Cloud Upload	Outbound	System → Supabase Backend	Anonymized metric and feedback records via HTTPS
Cloud Confirmation	Inbound	Supabase Backend → System	Upload confirmation (HTTP 200 OK response)

2.5 Design Decisions

The inclusion of Android System APIs as a distinct external entity is a deliberate design choice that reflects the technical reality of the QoE-Monitor system. Unlike typical application systems where the operating system is treated as transparent infrastructure, in QoE-Monitor the Android API layer is a genuine data source. TelephonyManager and ConnectivityManager are external to the application code and must be explicitly accessed through registered listeners and callbacks. Treating them as an external entity in the context diagram accurately represents the system's dependency on this data source and makes the data flows legible to both technical and non-technical stakeholders.

The separation of Supabase Backend and Network Operator into distinct entities, rather than treating them as a single cloud entity, reflects the different roles they play. Supabase is a technical infrastructure component that receives raw anonymized records. The Network Operator is an organizational actor that queries aggregated insights. These are fundamentally different interactions with different data characteristics and access control requirements.

3. Dataflow Diagram

3.1 Purpose and Scope

The Dataflow Diagram (DFD) extends the context diagram by opening the single system process and revealing its internal structure. Where the context diagram shows what data enters and exits the system, the DFD shows how that data moves through the system, which processes transform it, and where it is stored. The DFD presented here is a Level 0 diagram, which means it shows the major internal processes at one level of decomposition without drilling further into the sub-processes of each. This is the appropriate level of detail for system design documentation at this stage of the project.

The DFD uses a specific notation. Processes are represented as numbered circles or rounded rectangles. Data stores are represented as open-ended horizontal rectangles labelled with D1 and D2. External entities are the same rectangular boxes used in the context diagram. All data movements are represented as labelled directed arrows showing exactly what data flows in which direction.

3.2 External Entities

Three external entities appear in the Level 0 DFD, consistent with the context diagram. The Mobile Subscriber interacts with both data collection processes and receives output from the analytics process. Android System APIs feed raw technical data into the network metrics collection process. The Network Operator receives output exclusively from the analytics process.

3.3 Data Stores

Two data stores are defined in the system. D1 represents the Local Room Database, which is the encrypted SQLite database resident on the subscriber's Android device. This store holds

all NetworkMetric records, UserFeedback records, AppConfiguration data and synchronization status flags for every collected record. D1 is the central hub of the offline-first architecture, receiving data from collection processes and supplying data to the synchronization and dashboard processes.

D2 represents the Supabase Cloud Database, the backend storage hosted on Supabase's cloud infrastructure. This store holds all uploaded anonymized records and the aggregated SQL views through which operators access insights.

D2 is populated exclusively by the Data Synchronization process and is consumed exclusively by the Analytics and Dashboard process.

3.4 Internal Processes

Five internal processes collectively define the complete operational behaviour of the QoE-Monitor system at the Level 0 decomposition.

Process 1, Collect Network Metrics, is the core passive monitoring process. It receives data from Android System APIs through registered listeners and periodic WorkManager tasks. TelephonyManager provides signal strength values in dBm and network type identifiers. ConnectivityManager provides connectivity state changes. The process also performs its own latency measurement by executing lightweight HTTP round-trip timing requests to a measurement endpoint. All raw readings are packaged as NetworkMetric records and sent to D1 for local storage.

Process 2, Collect User Feedback, is the active data collection process. It receives feedback ratings and optional text comments from the Mobile Subscriber, either in response to an event-triggered notification prompt or through a manual report submission initiated by the subscriber. The process packages these ratings as UserFeedback records and links them to the NetworkMetric record that triggered the prompt using a foreign key relationship. Records are sent to D1 for storage.

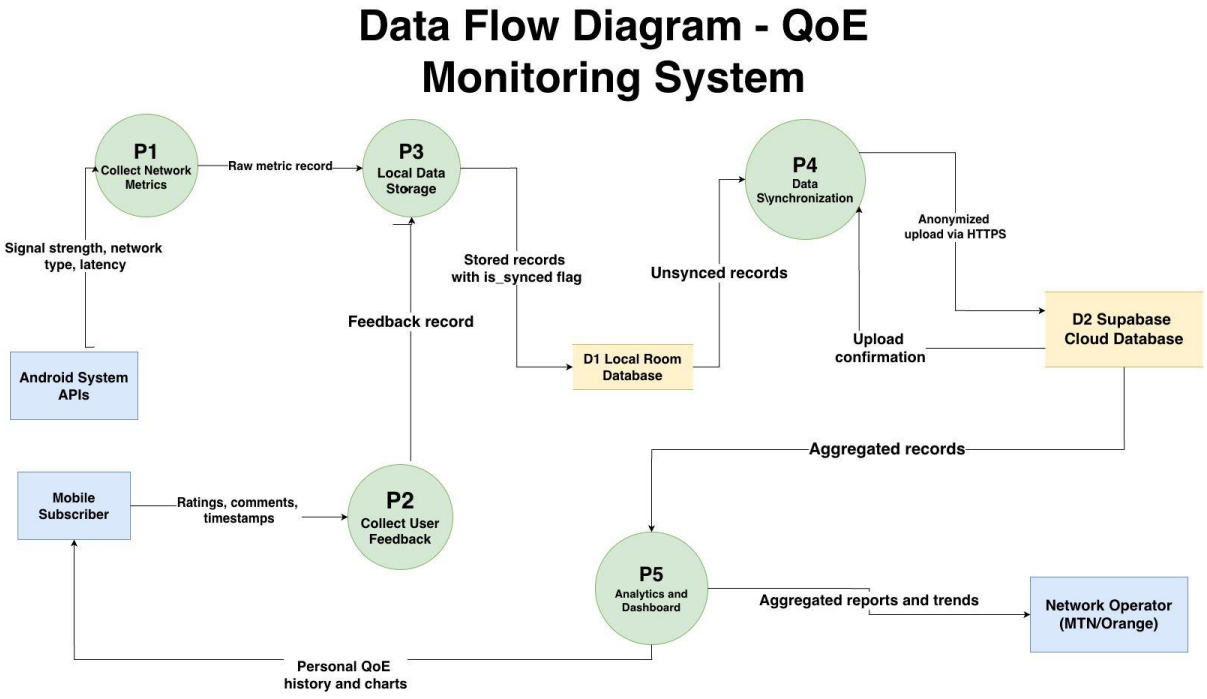
Process 3, Local Data Storage, manages the persistence lifecycle of all records in D1. It receives records from both P1 and P2, applies the is_synced flag set to false, and ensures records are correctly indexed by timestamp. It also implements the data retention policy,

scheduling deletion of synced records older than seven days and unsynced records older than thirty days. This process is the guardian of data integrity on the device.

Process 4, Data Synchronization, is responsible for moving data from the device to the cloud. It monitors network connectivity and activates when WiFi is available, or mobile data if the user has enabled that option. It fetches all unsynced records from D1, applies an anonymization transformation that removes all personally identifiable fields, and transmits the cleaned records to D2 via HTTPS REST API. On receiving a successful confirmation from D2, it updates the is_synced flags in D1. On failure, it implements exponential backoff retry logic.

Process 5, Analytics and Dashboard, serves data consumers on both sides of the system. For the Mobile Subscriber, it reads historical records from D2 and D1, generates the personal QoE trend chart and recent event history for the dashboard screen. For the Network Operator, it queries the aggregated Supabase views in D2 that expose average satisfaction scores, network type distributions, peak degradation periods and complaint trend patterns. Individual subscriber records are never surfaced to operators through this process.

3.4.1 Data Flow Diagram



3.5 Data Flow Summary

From	Process	To	Data Flow Label
Android APIs	P1 Collect Network Metrics	D1 Local Database	Signal strength, network type, latency → Raw metric record
Mobile Subscriber	P2 Collect User Feedback	D1 Local Database	Ratings, comments → Feedback record
D1 Local Database	P3 Local Data Storage	D1 Local Database	Record lifecycle management, retention cleanup
D1 Local Database	P4 Data Synchronization	D2 Supabase	Unsynced records → Anonymized upload
D2 Supabase	P4 Data Synchronization	D1 Local Database	Upload confirmation → Sync flag update
D2 Supabase	P5 Analytics and Dashboard	Mobile Subscriber	Aggregated records → Personal QoE dashboard
D2 Supabase	P5 Analytics and Dashboard	Network Operator	Aggregated views → QoE reports and trends

4. Use Case Diagram

4.1 Purpose and Scope

The Use Case Diagram defines all interactions between users and the QoE-Monitor system from the user's perspective. It describes what the system does for each type of user without specifying how it does it. This diagram is a key communication tool between the development team, stakeholders and end users because it presents system functionality in plain, non-technical language. The system boundary is represented as a large rectangle labelled QoE Monitor Application. All use cases appear inside this rectangle as labelled ovals. Actors appear outside the rectangle as named stick figures connected to their relevant use cases by association lines.

4.2 Actors

The QoE-Monitor system has four distinct actor types. The Mobile Subscriber is the primary actor who drives the most use cases. This actor represents any Android smartphone user in Cameroon who installs the application to monitor their network experience. The Android System is a secondary actor that represents the automated behavior of the phone's operating system — it fires events, schedules tasks and delivers notifications without any direct human action. The Network Operator is a secondary actor representing MTN or Orange Cameroon staff who access aggregated analytics. The Administrator is a secondary actor representing the backend system manager responsible for database integrity and access control.

4.3 Subscriber Use Cases

The Mobile Subscriber has nine primary use cases within the system. The Complete Onboarding use case covers the first launch experience in which the subscriber reads the data collection policy, accepts the terms, and configures initial settings. The Grant Permissions use case handles the subscriber's response to Android permission dialogs for READ_PHONE_STATE, ACCESS_FINE_LOCATION and POST_NOTIFICATIONS. The Configure Trigger Settings use case allows the subscriber to navigate to the Settings screen and individually enable or disable each event trigger type. The Respond to Feedback Prompt use case represents the core active feedback interaction, where the subscriber taps a notification and submits satisfaction ratings on the prompted feedback screen. The Submit Manual QoE Report use case allows the subscriber to initiate a feedback submission independently at any time using the manual report button. The View QoE Dashboard use case covers the subscriber's access to their personal network experience history and trend charts. The View QoE History use case allows filtering of the historical record by time periods. The Pause Monitoring use case allows the subscriber to temporarily halt all background data collection. The Delete Local Data use case allows the subscriber to permanently remove all locally stored records from the device.

4.4 Android System Use Cases

The Android System actor has five use cases that represent automated system-level behaviours. The Collect Network Metrics use case represents the periodic and event-driven background collection of signal strength, network type and latency data. The Detect Network Degradation use case represents the EventDetector evaluating incoming metrics against defined thresholds to identify signal drops, network type downgrades or latency spikes. The Detect Call End use case represents PhoneStateListener identifying the transition from an active call to idle state. The Schedule Periodic Collection use case represents WorkManager firing its registered background task at the configured interval. The Sync Data to Cloud use case represents ConnectivityManager detecting WiFi availability and triggering the SyncManager upload cycle.

4.5 Operator and Administrator Use Cases

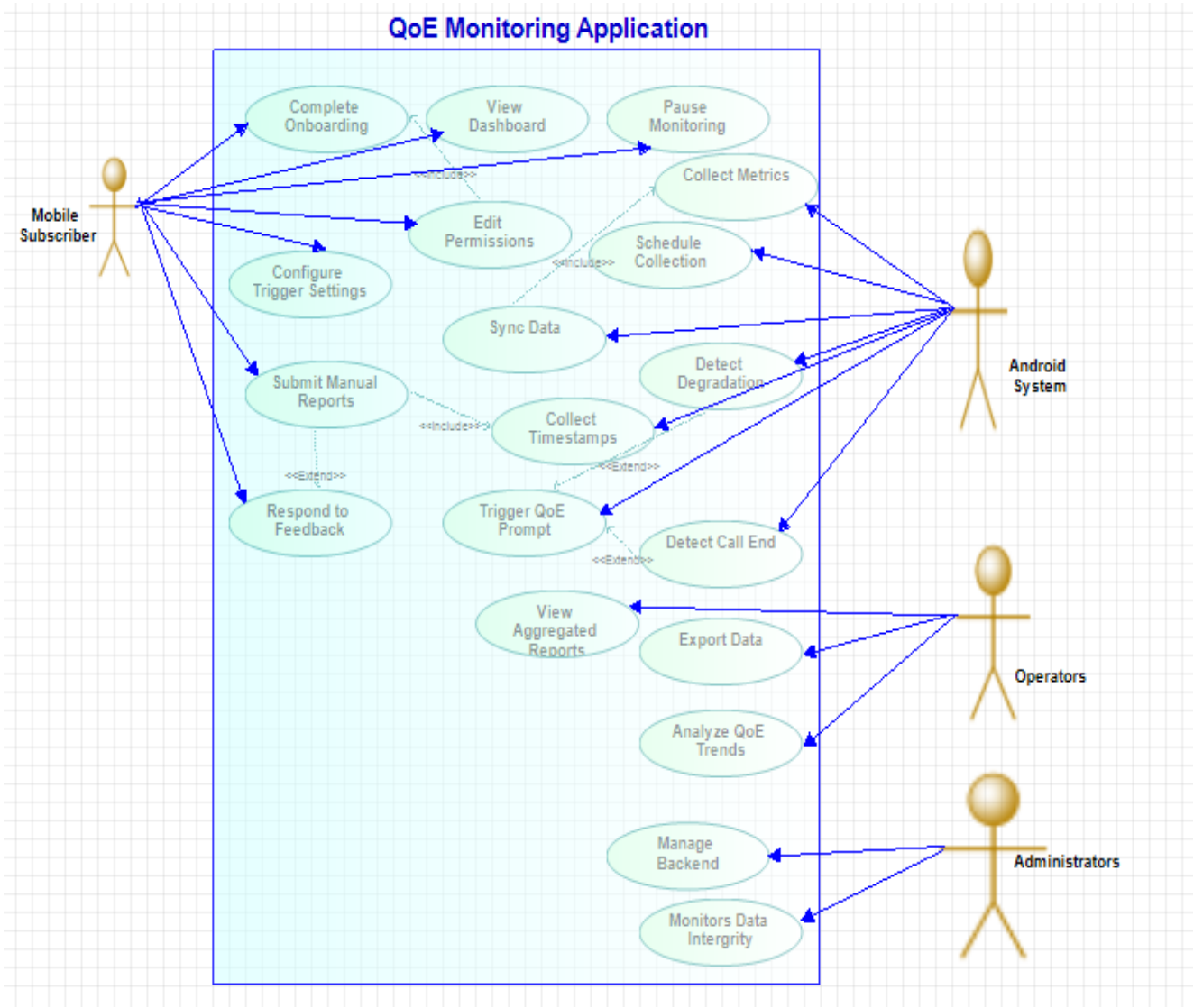
The Network Operator has three use cases. View Aggregated Reports covers the operator's access to the Supabase analytics dashboard showing QoE summaries. Analyze QoE Trends covers the operator's exploration of time-series patterns and degradation correlations. Export Analytics Data covers the operator's ability to download aggregated anonymized data as a CSV file for use in internal planning tools. The Administrator has two use cases. Manage Backend covers database administration, table management and access policy configuration in Supabase. Monitor Data Integrity covers the administrator's review of upload completion rates and data consistency checks.

4.6 Use Case Relationships

Several important relationships exist between use cases. The include relationship indicates that one use case always incorporates the behaviour of another. Complete Onboarding includes Grant Permissions because the permission request flow is an inseparable part of onboarding. Submit QoE Feedback includes Collect Timestamp because every feedback record always carries a timestamp. Sync Data to Cloud includes Collect Network Metrics because synchronization can only occur after collection has produced records to upload.

The extend relationship indicates that one use case optionally adds behaviour to another under specific conditions. Detect Degradation extends Trigger QoE Prompt because a feedback prompt is only sent if degradation thresholds are actually exceeded, not on every metric collection. Detect Call End extends Trigger QoE Prompt because the post-call prompt only fires when a call actually ends. Submit Manual Report extends Respond to Feedback Prompt because the manual submission flow is an optional extension of the standard feedback interaction.

4.6.1 Use Case Diagram



4.7 Use Cases Summary Table

Use Case ID	Use Case Name	Actor	Type
UC-01	Complete Onboarding / Register	Mobile Subscriber	Primary
UC-02	Grant Permissions	Mobile Subscriber	Primary
UC-03	Configure Trigger Settings	Mobile Subscriber	Primary
UC-04	Respond to Feedback Prompt	Mobile Subscriber	Primary
UC-05	Submit Manual QoE Report	Mobile Subscriber	Primary
UC-06	View QoE Dashboard	Mobile Subscriber	Primary
UC-07	View QoE History	Mobile Subscriber	Primary
UC-08	Pause Monitoring	Mobile Subscriber	Primary
UC-09	Delete Local Data	Mobile Subscriber	Primary
UC-10	Collect Network Metrics	Android System	Automated
UC-11	Detect Network Degradation	Android System	Automated
UC-12	Detect Call End	Android System	Automated
UC-13	Schedule Periodic Collection	Android System	Automated
UC-14	Sync Data to Cloud	Android System	Automated
UC-15	View Aggregated Reports	Network Operator	Secondary
UC-16	Analyze QoE Trends	Network Operator	Secondary
UC-17	Export Analytics Data	Network Operator	Secondary
UC-18	Manage Backend	Administrator	Secondary
UC-19	Monitor Data Integrity	Administrator	Secondary

5. Sequence Diagram

5.1 Purpose and Scope

The Sequence Diagram is a UML interaction diagram that shows how system components communicate with each other in a defined time order for a specific operational scenario. Unlike the use case diagram, which shows what the system does, and the class diagram, which shows how it is structured, the sequence diagram shows when things happen and in what sequence. Time flows vertically downward. Earlier messages appear higher on the diagram. Each component is represented as a vertical lifeline with a labelled box at the top. Messages between components are horizontal arrows connecting the lifelines, labelled with the action or data being communicated. Self-calls, where a component sends a message to itself, are shown as small loops on the same lifeline.

Two sequence diagrams are presented for QoE-Monitor, covering the two most significant operational flows: the event-triggered feedback collection flow, which represents the primary user-visible interaction of the system, and the background periodic monitoring flow, which represents the core automated data collection behaviour.

5.2 Sequence Diagram 1: Event-Triggered Feedback Collection

5.2.1 Scenario Description

This scenario describes the complete sequence of events that occurs when the Android operating system detects a signal drop on the subscriber's device. The scenario covers the full lifecycle from detection through metric collection, user notification, feedback submission, local storage and cloud synchronization. This is the most academically significant scenario because it demonstrates the integration of all major system components working together in response to a real network event.

5.2.2 Lifelines

Six lifelines participate in this scenario. The Subscriber lifeline represents the human user. The Mobile App UI lifeline represents the visible screen layer of the application. The Monitoring Service lifeline represents the background Foreground Service running the monitoring logic. The Android APIs lifeline represents TelephonyManager, ConnectivityManager and related system services. The Room Database lifeline represents the local SQLite database on the device. The Supabase Backend lifeline represents the cloud server.

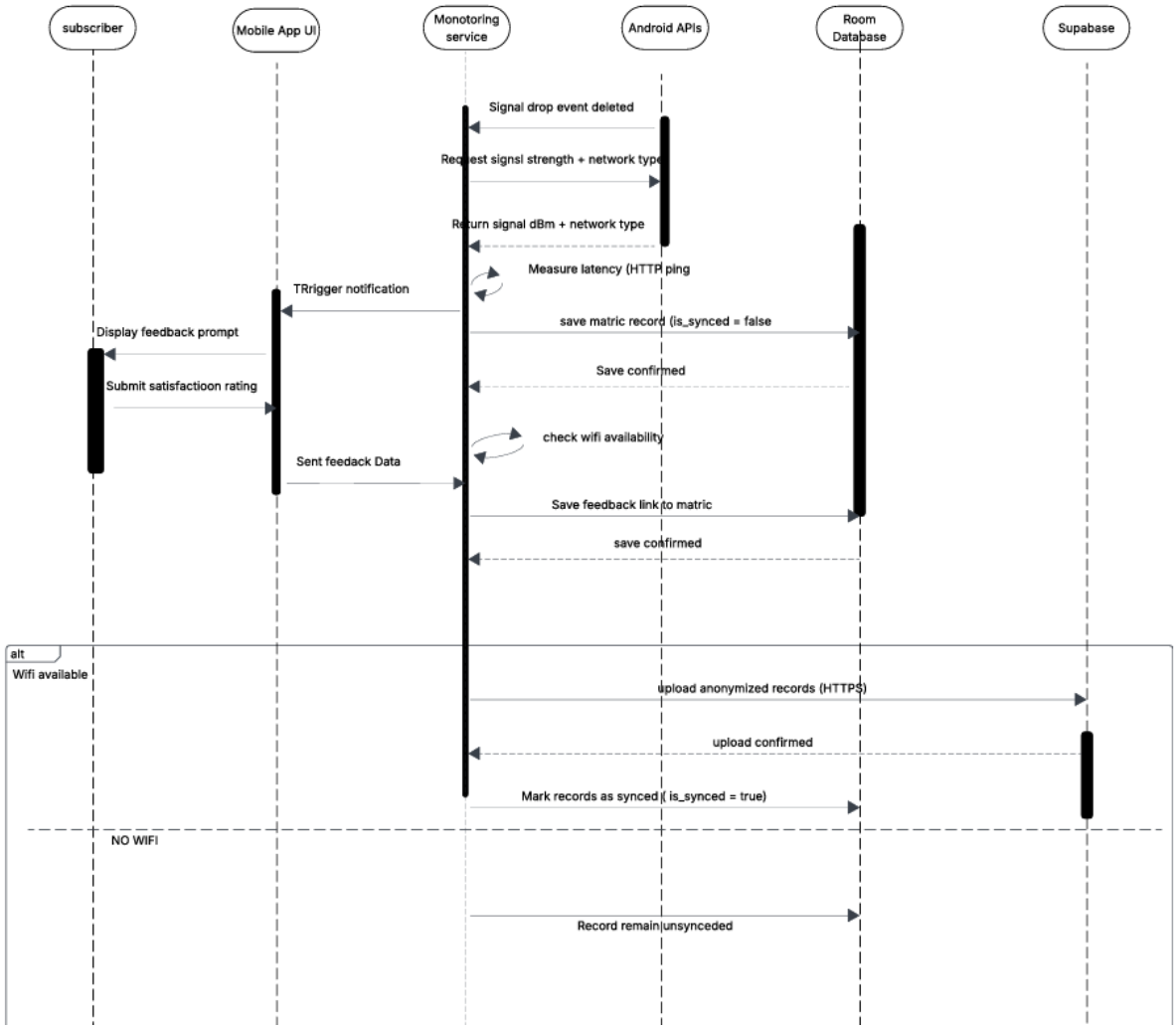
5.2.3 Message Sequence

Step	From	To	Message
1	Android APIs	Monitoring Service	Signal drop event broadcast received (BroadcastReceiver fires)
2	Monitoring Service	Android APIs	Request: getSignalStrength() and getNetworkType()
3	Android APIs	Monitoring Service	Return: signal strength in dBm and network type (e.g. 3G)
4	Monitoring Service	Monitoring Service	Self-call: measure latency via HTTP ping (Retrofit)
5	Monitoring Service	Room Database	INSERT NetworkMetric record (is_synced = false)
6	Room Database	Monitoring Service	Save confirmed — record ID returned
7	Monitoring Service	Mobile App UI	Trigger local notification: Your network seems slow — how is your experience?
8	Mobile App UI	Subscriber	Display feedback prompt on status bar
9	Subscriber	Mobile App UI	Taps notification — opens feedback screen
10	Mobile App UI	Subscriber	Display feedback form (overall satisfaction, browsing experience)
11	Subscriber	Mobile App UI	Submits ratings
12	Mobile App UI	Monitoring Service	Send: feedback data with ratings
13	Monitoring Service	Room Database	INSERT UserFeedback record linked to NetworkMetric via networkMetricId

Step	From	To	Message
14	Room Database	Monitoring Service	Save confirmed
15	Monitoring Service	Monitoring Service	Self-call: check — is WiFi available?
16 [WiFi available]	Monitoring Service	Room Database	SELECT all records WHERE is_synced = false
17	Room Database	Monitoring Service	Return unsynced records
18	Monitoring Service	Monitoring Service	Self-call: anonymize records — strip PII fields
19	Monitoring Service	Supabase Backend	POST anonymized batch via HTTPS REST
20	Supabase Backend	Monitoring Service	200 OK — upload confirmed
21	Monitoring Service	Room Database	UPDATE is_synced = true for uploaded records
22 [No WiFi]	Monitoring Service	Room Database	Records remain unsynced — retry at next scheduled cycle

Steps 16 through 22 are enclosed in an alt combined fragment with two conditions: the upper partition labelled WiFi available covering steps 16 through 21, and the lower partition labelled No WiFi covering step 22. This fragment makes the conditional synchronization logic explicit in the diagram.

5.2.4 Sequence Diagram For Event Triggered Feedback Collection



5.3 Sequence Diagram 2: Background Periodic Monitoring

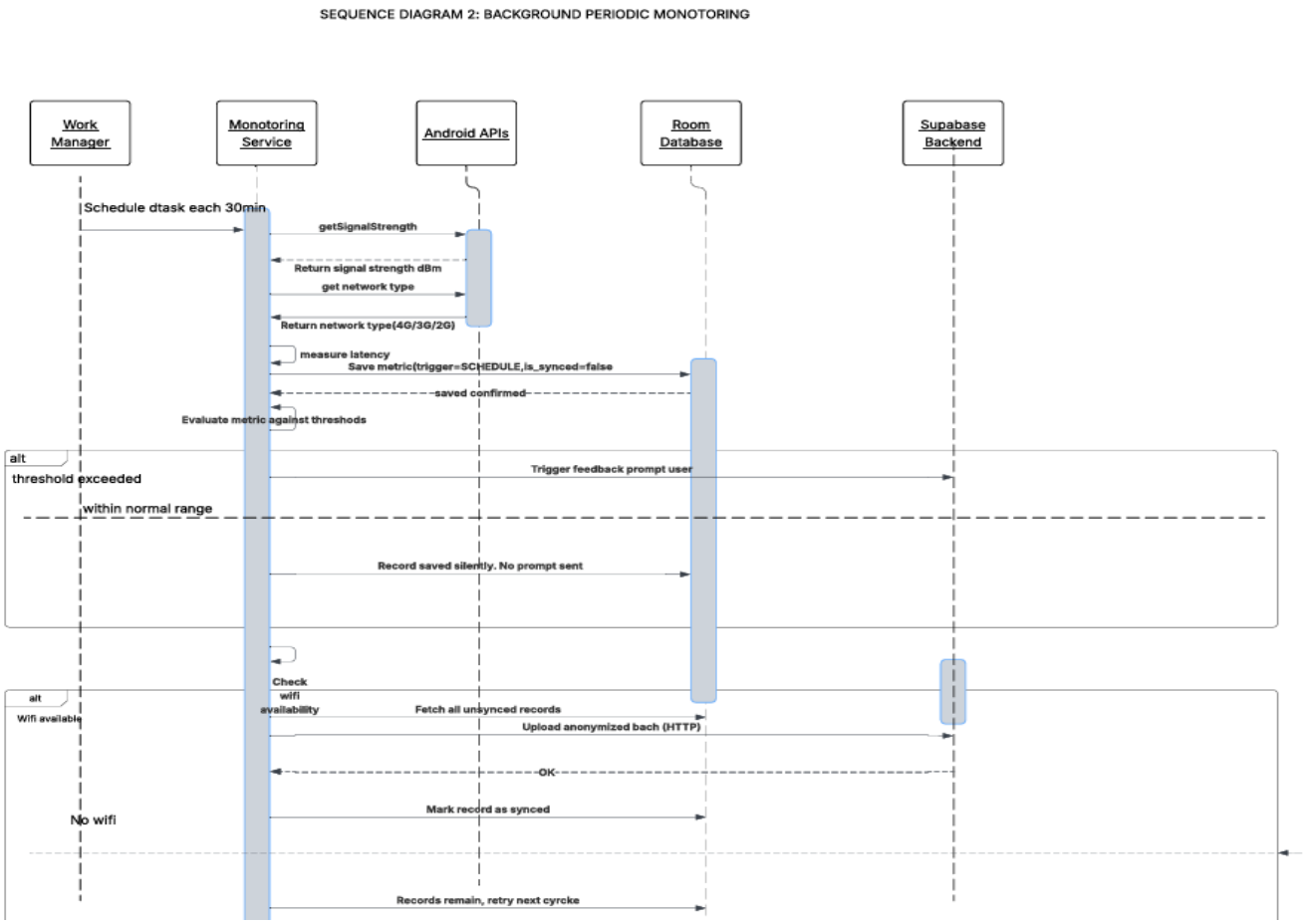
5.3.1 Scenario Description

This scenario describes the automated background monitoring cycle that occurs without any user interaction. WorkManager fires a scheduled task at regular intervals, typically every thirty minutes. The application collects network metrics, evaluates them against defined thresholds, stores the results locally, and synchronizes to the cloud if WiFi is available. If a threshold is exceeded during this cycle, a feedback prompt is triggered. This scenario demonstrates the passive, always-on character of the monitoring system.

5.3.2 Message Sequence

Step	From	To	Message
1	WorkManager	Monitoring Service	Scheduled task fires (interval: every 30 minutes)
2	Monitoring Service	Android APIs	getSignalStrength()
3	Android APIs	Monitoring Service	Return: signal strength in dBm
4	Monitoring Service	Android APIs	getNetworkType()
5	Android APIs	Monitoring Service	Return: network type (4G / 3G / 2G)
6	Monitoring Service	Monitoring Service	Self-call: measure latency via HTTP ping
7	Monitoring Service	Room Database	INSERT NetworkMetric (trigger = SCHEDULED, is_synced = false)
8	Room Database	Monitoring Service	Save confirmed
9	Monitoring Service	Monitoring Service	Self-call: evaluate metrics against thresholds
10 [threshold exceeded]	Monitoring Service	Mobile App UI	Trigger feedback prompt notification to user
11 [within normal range]	Monitoring Service	Room Database	Record saved silently — no prompt sent
12	Monitoring Service	Monitoring Service	Self-call: check connectivity — WiFi available?
13 [WiFi]	Monitoring Service	Room Database	Fetch all unsynced records
14	Monitoring Service	Monitoring Service	Self-call: anonymize records
15	Monitoring Service	Supabase Backend	POST anonymized batch
16	Supabase Backend	Monitoring Service	200 OK confirmation
17	Monitoring Service	Room Database	UPDATE is_synced = true
18 [No WiFi]	Monitoring Service	Room Database	Records remain — retry next cycle

5.3.3 Sequence Diagram for Periodic Monitoring



This sequence diagram contains two alt fragments. The first covers steps 10 and 11, with partitions labelled threshold exceeded and within normal range. The second covers steps 13 through 18, with partitions labelled WiFi available and No WiFi. These fragments make explicit the conditional logic that governs both the feedback trigger decision and the synchronization decision.

6. Class Diagram

6.1 Purpose and Scope

The Class Diagram is the primary structural model of the QoE-Monitor system. It defines all classes, their attributes with data types, their methods, and the relationships between them including multiplicity and inheritance. The diagram is organized into three conceptual layers following the MVVM and Clean Architecture pattern: Presentation at the top, Domain in the middle, and Data at the base. The diagram also introduces an inheritance hierarchy under the User class, reflecting the three distinct actor types that interact with the system: Subscriber, Operator and Administrator.

6.2 Inheritance Hierarchy: User and Actor Subclasses

The User class serves as the parent superclass representing any person who interacts with the QoE-Monitor system. All common identity attributes and base behaviours shared across actor types are defined here. Three child classes extend User through an inheritance relationship, each specializing the parent with role-specific attributes and methods.

The Subscriber class extends User and represents the mobile phone user who installs the app, grants permissions, responds to feedback prompts and views their personal QoE dashboard. It adds deviceModel of type String and androidVersion of type String as attributes, and the methods viewDashboard(), deleteLocalData() and configureTriggers(). Since Subscriber inherits from User, it automatically possesses userId, monitoringEnabled and preferences without these being redeclared.

The Operator class extends User and represents MTN or Orange Cameroon staff who access the aggregated analytics backend. It adds organizationName of type String and accessLevel of type String, and the methods viewAggregatedReports(), exportAnalytics() and analyzeQoETrends(). Operators interact exclusively with the cloud backend and never directly with subscriber devices.

The Administrator class extends User and represents the system manager responsible for Supabase database administration and access control. It adds adminRole of type String and the methods manageBackend(), monitorDataIntegrity() and manageAccessPolicies().

In UML, inheritance is drawn as a solid line with a hollow triangle arrowhead pointing from the child class to the parent. All three child classes point their hollow triangle up toward User.

6.3 Data Layer Classes

The NetworkMetric class is a data entity representing one collected network performance reading. Its attributes are: recordId of type String as the unique UUID primary key, timestamp of type Long, signalStrengthDbm of type Integer, networkType of type String, latencyMs of type Integer, triggerEvent of type String, and isSynced of type Boolean. Its method is collectMetrics().

The QoEFeedback class represents one user feedback submission. Its attributes are: feedbackId of type String, rating of type Integer, comment of type String, timestamp of type Long, networkMetricId of type String as the foreign key linking to the triggering NetworkMetric, and isSynced of type Boolean. Its method is saveFeedback().

The AppConfiguration class stores persistent application state. Its attributes are: userId of type String, activeTriggers of type String storing JSON-encoded trigger preferences, onboardingComplete of type Boolean, syncOnWifiOnly of type Boolean, and lastSyncTime of type Long. This class has no methods as it is a pure data container. Because it is associated with the User parent class, all three actor subclasses inherit access to configuration data.

The QoERepository class is the single gateway between all other classes and the local database. Its attributes are metricDao of type NetworkMetricDao, feedbackDao of type UserFeedbackDao, and configDao of type AppConfigurationDao. Its methods are saveMetric(), saveFeedback(), getUnsyncedRecords(), getDashboardData(), getConfig() and updateConfig().

6.4 Domain Layer Classes

The NetworkMonitoringService class is an Android Foreground Service running persistently in the background. Its attributes are telephonyManager of type TelephonyManager, connectivityManager of type ConnectivityManager, and feedbackManager of type FeedbackManager. Its methods are startMonitoring(), stopMonitoring(), collectMetrics() and measureLatency(). It maintains a composition relationship with NetworkMetric, creating and owning all metric records. It also carries a new association with Subscriber, reflecting that monitoring runs on behalf of a specific subscriber’s device.

The EventDetector class evaluates incoming NetworkMetric records against defined thresholds. Its attributes are signalThreshold of type Integer and latencyThreshold of type Integer. Its methods are checkSignalDrop(), checkNetworkDowngrade(), checkLatencySpike() and evaluateMetric(). When a threshold is exceeded, EventDetector notifies FeedbackManager through a dependency relationship — this arrow was previously missing from the diagram and has been added.

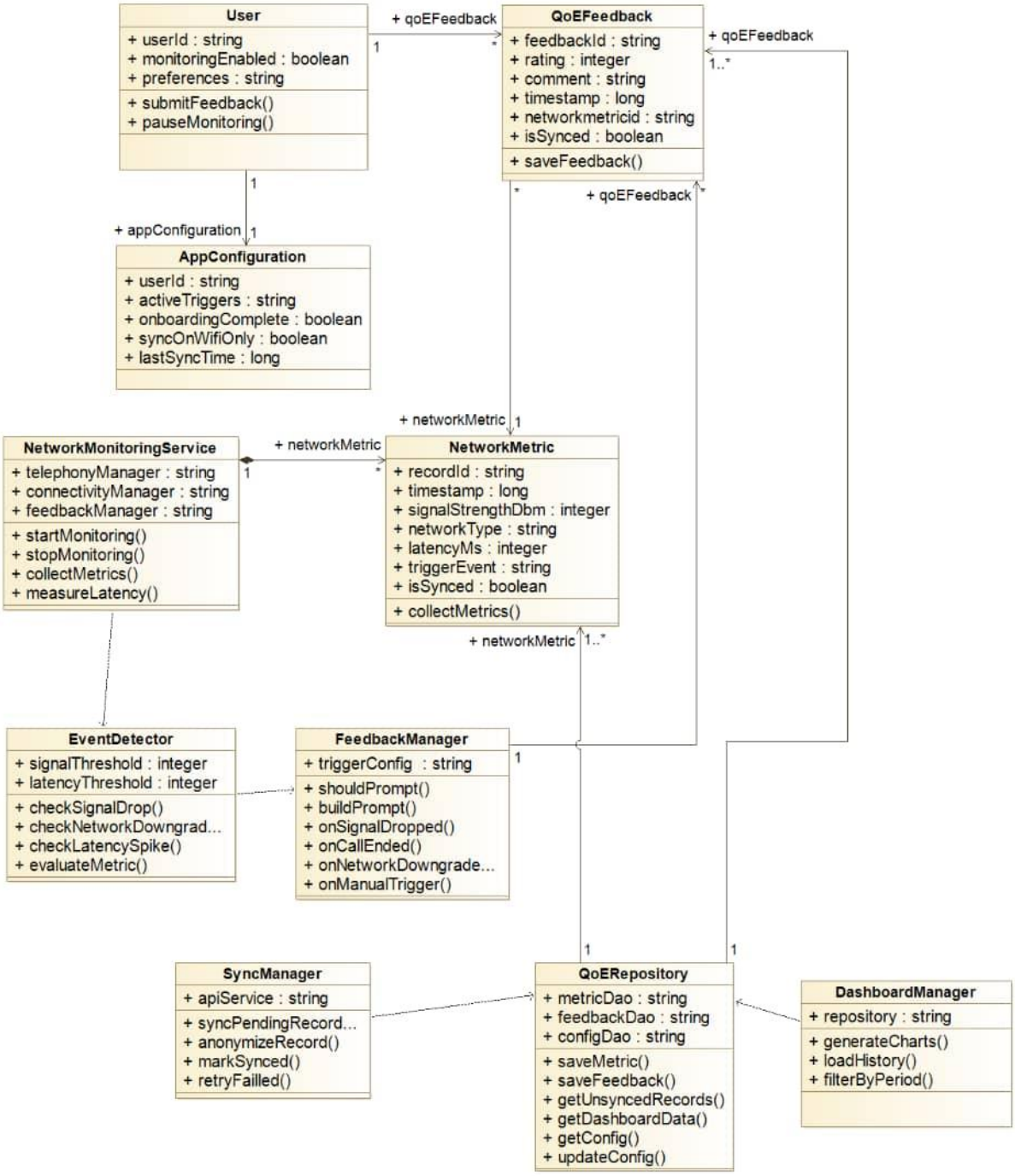
The FeedbackManager class decides when and how to prompt the subscriber. Its attribute is triggerConfig of type AppConfiguration. Its methods are shouldPrompt(), buildPrompt(), onSignalDropped(), onCallEnded(), onNetworkDowngrade() and onManualTrigger().

The SyncManager class manages the complete upload lifecycle. Its attribute is apiService of type SupabaseApiService. Its methods are syncPendingRecords(), anonymizeRecord(), markSynced() and retryFailed(). SyncManager depends on QoERepository to fetch unsynced records — this dependency arrow was previously missing and has been added.

6.5 Presentation Layer Classes

The DashboardManager class drives the QoE history and analytics screens. Its attribute is repository of type QoERepository. Its methods are generateCharts(), loadHistory() and filterByPeriod(). DashboardManager now carries two explicit association relationships reflecting that it serves two distinct audiences: one association to Subscriber for the personal QoE history view, and one association to Operator for the aggregated analytics view. Both associations are drawn in the updated diagram.

6.5.1 Class Diagram



6.6 Complete Class and Relationship Summary

From	To	Relationship	Multiplicity	Description
Subscriber	User	Inheritance	IS-A	Subscriber is a specialized User with device attributes
Operator	User	Inheritance	IS-A	Operator is a specialized User with analytics access
Administrator	User	Inheritance	IS-A	Administrator is a specialized User with backend management
User	AppConfiguration	Association	1 to 1	Every actor type has exactly one configuration (inherited)
Subscriber	QoEFeedback	One-to-Many	1 to *	Only Subscribers submit feedback — moved from User
QoEFeedback	NetworkMetric	Association	* to 1	Each feedback links to one triggering metric via networkMetricId
NetworkMonitoringService	Subscriber	Association	1 to 1	Service monitors on behalf of a specific subscriber device
NetworkMonitoringService	NetworkMetric	Composition	1 to *	Service creates and owns all metric records
NetworkMonitoringService	EventDetector	Dependency	—	Service passes metrics to detector for threshold evaluation
EventDetector	FeedbackManager	Dependency	—	Detector notifies manager when threshold crossed (was missing)
FeedbackManager	QoERepository	Association	1 to 1	Manager saves feedback through the repository
SyncManager	QoERepository	Dependency	—	Sync fetches unsynced records through repository (was missing)
DashboardManager	QoERepository	Dependency	—	Dashboard reads all data through the repository
DashboardManager	Subscriber	Association	1 to 1	Serves personal QoE history to subscriber

From	To	Relationship	Multiplicity	Description
DashboardManager	Operator	Association	1 to 1	Serves aggregated analytics reports to operator
QoERepository	NetworkMetric	Manages	1 to *	Repository performs all CRUD on metric records
QoERepository	QoEFeedback	Manages	1 to *	Repository performs all CRUD on feedback records

7. Deployment Diagram

7.1 Purpose and Scope

The Deployment Diagram shows the physical architecture of the QoE-Monitor system by mapping software components to the hardware and infrastructure nodes on which they are deployed. Where the class diagram shows the logical structure of the software and the sequence diagram shows how components interact over time, the deployment diagram shows where each component lives in the physical world and how those physical locations communicate. This diagram is particularly important for QoE-Monitor because the system is fundamentally distributed.

It spans a subscriber's Android device, the Android operating system layer, a cloud database server, and an operator's access point. Understanding how these physical environments connect is essential for security planning, performance optimization and deployment strategy.

7.2 Node 1: Subscriber Android Device

The subscriber's Android smartphone is the primary deployment node of the system. It hosts the QoE-Monitor APK, which is the compiled application package containing all application code. Within the application, several distinct runtime components are deployed. The Foreground Service is a persistent background process that keeps the monitoring engine alive by displaying a system notification as required by Android policy. WorkManager Tasks are

scheduled background jobs managed by Android's WorkManager framework, responsible for periodic metric collection and synchronization retry logic. The BroadcastReceiver is a registered component that listens for system-wide connectivity events. The Jetpack Compose UI layer provides all screen components including the dashboard, feedback forms, settings screen and onboarding flow. The Room Database is an encrypted SQLite database stored in the application's private internal storage, inaccessible to other applications on the same device.

The physical characteristics of this node are significant for the system's design decisions. The dominant devices in the Cameroonian market are Tecno and Infinix smartphones, both of which run aggressive battery optimization policies that are more restrictive than stock Android. These policies are known to terminate background processes more aggressively than the Android specification requires. The hybrid event-based monitoring architecture is a direct response to this reality, minimizing the background activity surface area to reduce the likelihood of system termination.

7.3 Node 2: Android System Services

Android System Services represent a distinct logical node that, while physically collocated with the subscriber's device, operates at a different software layer — the Android operating system kernel and framework layer. This node exposes the APIs through which the QoE-Monitor application accesses hardware data. TelephonyManager provides access to the cellular radio and exposes signal strength in dBm and network type identifiers. ConnectivityManager monitors the active network connection state and fires broadcasts when connectivity changes occur.

NotificationManager delivers local notifications to the device's status bar. The WorkManager Scheduler is the Android framework component that manages the lifecycle of background task execution, including scheduling, retry logic and boot persistence.

The communication between Node 1 and Node 2 occurs through the Android SDK's internal inter-process communication mechanisms. No network connection is required. This internal communication link is labelled Android SDK — Internal in the deployment diagram.

7.4 Node 3: Supabase Cloud Backend

The Supabase Cloud Backend is a server-side node hosted on Supabase's managed PostgreSQL infrastructure. It consists of multiple deployed components. The Supabase REST API Layer is the HTTP interface through which the Android device uploads records, authenticates sessions and queries data.

The PostgreSQL Database stores all received anonymized records in three main tables: the NetworkMetrics table, the UserFeedback table, and the AppConfiguration reference table. Two Supabase SQL Views are defined on top of these tables and are the only data surfaces exposed to operators: an aggregated satisfaction view summarizing average scores by time period, and a network type distribution view showing the breakdown of connection types across all subscribers.

Row Level Security Policies are defined at the database level and enforce that operator database roles can only query the aggregated views, never the underlying record tables. Supabase Auth manages the authentication layer for operator and administrator access.

This node receives data exclusively from the Android Device node via HTTPS REST API and serves data exclusively to the Operator Access Point node via the same protocol. The two communication paths have fundamentally different access characteristics: the device path is write-only for records and the operator path is read-only for aggregated views.

7.5 Node 4: Operator Access Point

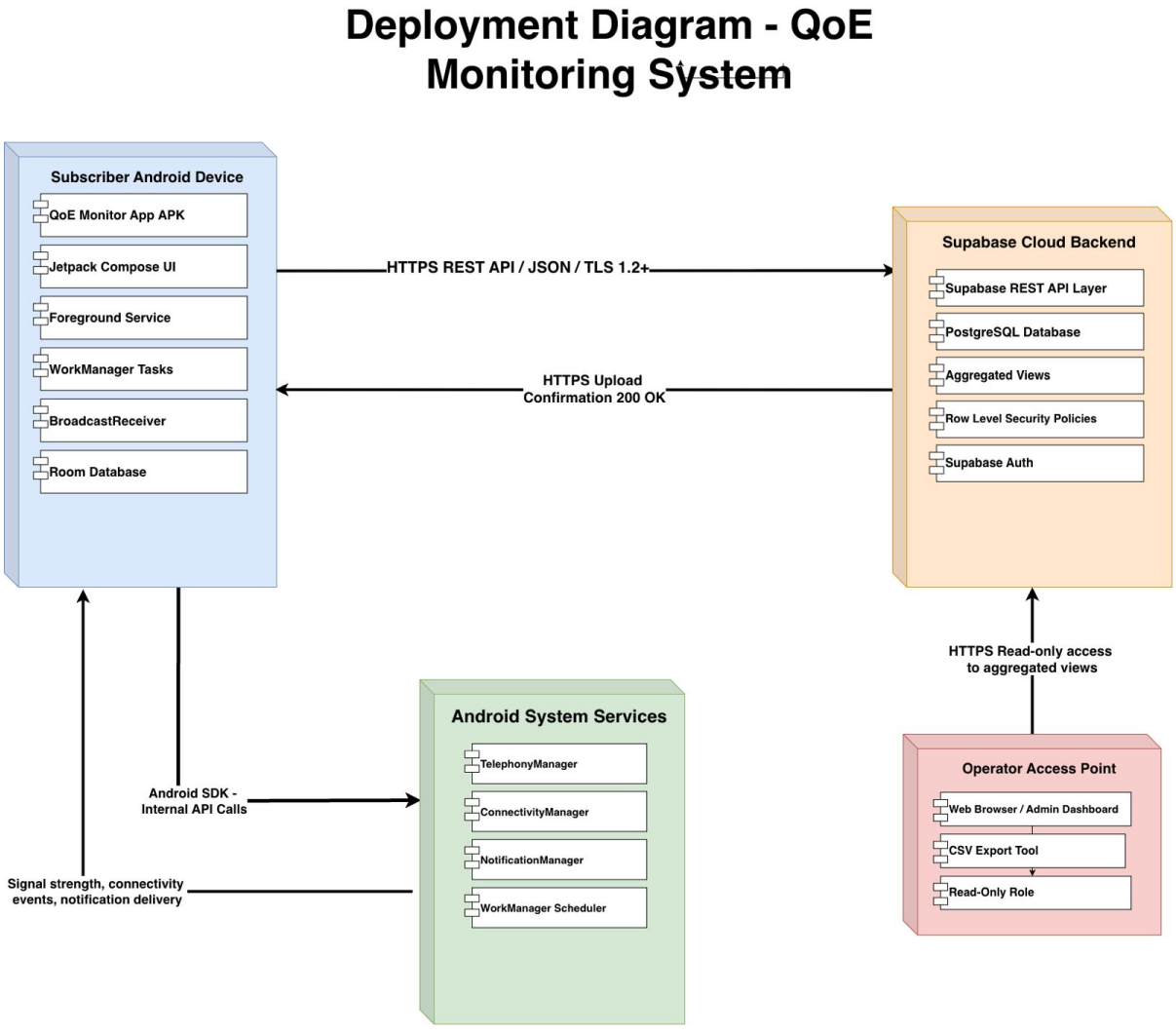
The Operator Access Point represents the hardware and software environment used by network operator staff to access QoE analytics. In practice this is a computer workstation running a web browser connected to the Supabase analytics dashboard. This node contains the web browser or dedicated admin dashboard application, a CSV export tool for downloading aggregated data in a format compatible with operator planning tools, and the read-only database role credential that limits all database interactions to SELECT queries on the aggregated views.

A critical architectural constraint that the deployment diagram makes visually explicit is that the Operator Access Point node connects only to the Supabase Cloud Backend node.

There is no direct communication path between the Operator Access Point and the Subscriber Android Device.

Operators access only what the cloud exposes to them through the aggregated views. They cannot reach raw subscriber records, and they have no pathway to the device at all. This physical separation of the operator from the subscriber device is the deployment-level implementation of the system's privacy architecture.

7.5.1 Deployment Diagram



7.6 Communication Links

From Node	To Node	Protocol	Direction
Android Device	Android System Services	Android SDK — Internal	Bidirectional
Android Device	Supabase Cloud Backend	HTTPS REST API / JSON / TLS 1.2+	Device initiates (upload)
Supabase Cloud Backend	Android Device	HTTPS — Upload Confirmation	Cloud responds
Operator Access Point	Supabase Cloud Backend	HTTPS — Read-Only	Operator initiates (query)

8. Design Decisions and Rationale

8.1 Native Kotlin over Cross-Platform Frameworks

The decision to implement QoE-Monitor as a native Android application using Kotlin rather than a cross-platform framework such as Flutter or React Native was driven by the specific technical requirements of the system's core monitoring functionality. The system requires direct access to TelephonyManager and ConnectivityManager APIs at the Android SDK level to read signal strength in dBm and detect network type. While cross-platform frameworks provide wrappers around many Android APIs, these wrappers introduce abstraction layers that reduce reliability and control, particularly for low-level telephony access and Foreground Service lifecycle management. Native Kotlin provides direct, unmediated access to every Android API, better Foreground Service lifecycle control, and improved performance for continuous background monitoring workloads. On Tecno and Infinix devices, which use Android OEM firmware with aggressive battery optimization, native applications are terminated less readily than cross-platform runtimes.

iOS was excluded entirely from the platform scope. This decision is not a matter of preference but of technical impossibility. Apple does not expose TelephonyManager-equivalent telephony APIs to third-party applications. Signal strength readings and network type data are inaccessible to any third-party application on iOS, regardless of what programming language

or framework is used. No cross-platform solution can bridge this gap because the underlying API does not exist on the iOS platform for third-party use. The context diagram and deployment diagram reflect this constraint by showing only Android device nodes.

8.2 Offline-First Architecture

The decision to implement an Offline-First architecture, in which all data is persisted locally before any transmission is attempted, is directly motivated by the problem the system exists to solve. QoE-Monitor is designed to study network unreliability. It would be architecturally contradictory for the system to depend on reliable network connectivity to save its own data. When a subscriber's network degrades and a metric is collected, the network degradation itself may prevent immediate upload. Local persistence to Room Database guarantees that no data point is lost regardless of connectivity state. The synchronization to Supabase occurs opportunistically when connectivity is available, defaulting to WiFi to avoid consuming the subscriber's mobile data.

8.3 Hybrid Event-Based Monitoring Architecture

Continuous background polling which occurs whenwhere the application constantly reads metrics on a tight loop it is not viable on modern Android devices. Android's Doze mode and App Standby policies actively restrict background processes to conserve battery. Applications that attempt continuous background processing are terminated by the operating system. Additionally, continuous polling would appear in the device's battery usage statistics, causing users to uninstall the application. The hybrid event-based architecture resolves this tension. WorkManager handles infrequent scheduled checks at intervals of thirty minutes or more, which falls within Android's background execution windows. BroadcastReceiver handles real-time event detection without continuous polling, as it is invoked only when a relevant system event fires. The Foreground Service provides persistent operation only during active monitoring episodes. Together these mechanisms achieve the goal of continuous network awareness while respecting Android's operational constraints and the user requirement for minimal battery impact.

8.4 Anonymization Before Upload

The privacy architecture of the system enforces anonymization at the device level before any data transmission occurs. This means that personally identifiable fields such as phone number, IMEI, carrier account details, and real name, are never included in records stored for upload. The anonymous user UUID is generated using a cryptographically random UUID generator on first install and is never derived from any hardware identifier. This approach means that even if the Supabase backend were compromised, no personally identifiable information about any subscriber would be exposed because it was never uploaded. Operators access only aggregated views through Row Level Security policies, providing a second layer of privacy protection at the database level.

9. Conclusion

This report has presented the complete system modelling and design for QoE-Monitor, a native Android application designed to collect real-time Quality of Experience data from mobile network subscribers in Cameroon. The six diagrams produced during this phase collectively define the system from the highest level of external abstraction down to the structural detail of individual classes and their relationships.

The Context Diagram established that the system interacts with four external entities: the Mobile Subscriber, Android System APIs, the Network Operator and the Supabase Backend. The Dataflow Diagram decomposed the system into five internal processes, two data stores, and defined the complete data movement architecture including the Offline-First synchronization pattern. The Use Case Diagram catalogued nineteen use cases across four actor types, capturing the full behavioural scope of the system from the user perspective. The Sequence Diagrams demonstrated in time-ordered detail how the system’s components collaborate during the two most critical operational scenarios. The Class Diagram defined ten primary classes across three architectural layers which are; Presentation, Domain and Data with all attributes, methods, relationships and multiplicities specified. The Deployment Diagram mapped all software components to their physical nodes and defined the communication protocols and access constraints between them.

Together these diagrams validate that the system is architecturally sound, technically feasible within Android platform constraints, and correctly aligned with the functional and non-

functional requirements established in Task 3. The design decisions documented in Section 8 which are; native Kotlin over cross-platform frameworks, Offline-First architecture, hybrid event-based monitoring, and on-device anonymization before upload, each reflect specific technical, operational or regulatory constraints that were identified during the requirement gathering and analysis phases.

The next phase of the project, Task 5: UI Design and Implementation, will translate this architectural foundation into tangible user interface screens, visual design and frontend implementation. The class diagram and deployment diagram produced in this phase provide the structural blueprint that will guide database design in Task 6 and the final implementation across Tasks 5 and 6.

References

- Android Developers. TelephonyManager API Reference. https://developer.android.com/reference/android/telephony/TelephonyManager
- Android Developers. ConnectivityManager API Reference. https://developer.android.com/reference/android/net/ConnectivityManager
- Android Developers. WorkManager Guide. https://developer.android.com/topic/libraries/architecture/workmanager
- Android Developers. Background Processing Guide. https://developer.android.com/guide/background
- Android Developers. Foreground Services. https://developer.android.com/guide/components/foreground-services
- Supabase Documentation. Database and REST API. https://supabase.com/docs
- Object Management Group. Unified Modelling Language Specification, Version 2.5.1. OMG, 2017.
- IEEE Std 830-1998. IEEE Recommended Practice for Software Requirements Specifications. IEEE, 1998.
- Group 2. Task 2: Requirement Gathering Report. University of Buea, CEF440, 2026.
- Group 2. Task 3: Requirement Analysis Report and SRS. University of Buea, CEF440, 2026.